

Program 9: Maven To Gradle

Step 1: Create a new maven project using this command

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenToGradle -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
rakshith@rakshith-VirtualBox:~/Devops$ mvn archetype:generate.example -DartifactId=MyMavenToGradle -DarchetypeArtifactId=ma
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18 MB at 6.0 MB/s)
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/rakshith/Devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenToGradle
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/rakshith/Devops/MyMavenToGradle
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 8.727 s
[INFO] Finished at: 2026-03-25T08:57:28+05:30
[INFO]
rakshith@rakshith-VirtualBox:~/Devops$ █
```

Step 2: Edit Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.example1</groupId>
<artifactId>MyMavenToGradle</artifactId>
<packaging>jar</packaging>
<version>1.0-SNAPSHOT</version>
<name>MyMavenToGradle</name>
<url>http://maven.apache.org</url>
<!-- Properties: Customize Java version or plugin versions -->
<properties>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
```

```
</properties>
<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
</dependencies>
<!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.6</source>
<target>1.6</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-jar-plugin</artifactId>
<version>3.1.0</version>
<configuration>
<archive>
<manifestEntries>
<Main-Class>com.example1.App</Main-Class>
</manifestEntries>
</archive>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

Step 3: Execute the command mvn clean install:

```
rakshith@rakshith-VirtualBox:~/Devops/MyMavenToGradle$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example1:MyMavenToGradle >-----
[INFO] Building MyMavenToGradle 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenToGradle ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenToGradle ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/rakshith/Devops/MyMavenToGradle/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenToGradle ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/rakshith/Devops/MyMavenToGradle/target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenToGradle ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/rakshith/Devops/MyMavenToGradle/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenToGradle ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/rakshith/Devops/MyMavenToGradle/target/test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenToGradle ---
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s - in com.example.AppTest
[INFO]
```

Step 4: When the command ‘gradle init -type pom.xml’ is executed, we get a error

```
rakshith@rakshith-VirtualBox:~/Devops/MyMavenToGradle$ gradle init --type pom.xml

FAILURE: Build failed with an exception.

* What went wrong:
Execution failed for task ':init'.
> The requested build setup type 'pom.xml' is not supported. Supported: 'basic', 'groovy-application', '

* Try:
Run with --stacktrace option to get the stack trace. Run with --info or --debug option to get more log or

* Get more help at https://help.gradle.org

BUILD FAILED in 6s
2 actionable tasks: 2 executed
rakshith@rakshith-VirtualBox:~/Devops/MyMavenToGradle$ █
```

Step 5: execute 'gradle init -type pom' to get build success

```
rakshith@Ubuntu:~/Devops/MyMavenToGradle$ gradle init --type pom
```

```
> Task :init
```

```
Maven to Gradle conversion is an incubating feature.
```

```
BUILD SUCCESSFUL in 5s
```

```
2 actionable tasks: 1 executed, 1 up-to-date
```

```
rakshith@Ubuntu:~/Devops/MyMavenToGradle$
```

Step 6: Display tree structure

```
rakshith@rakshith-VirtualBox:~/Devops/MyMavenToGradle$ tree
```

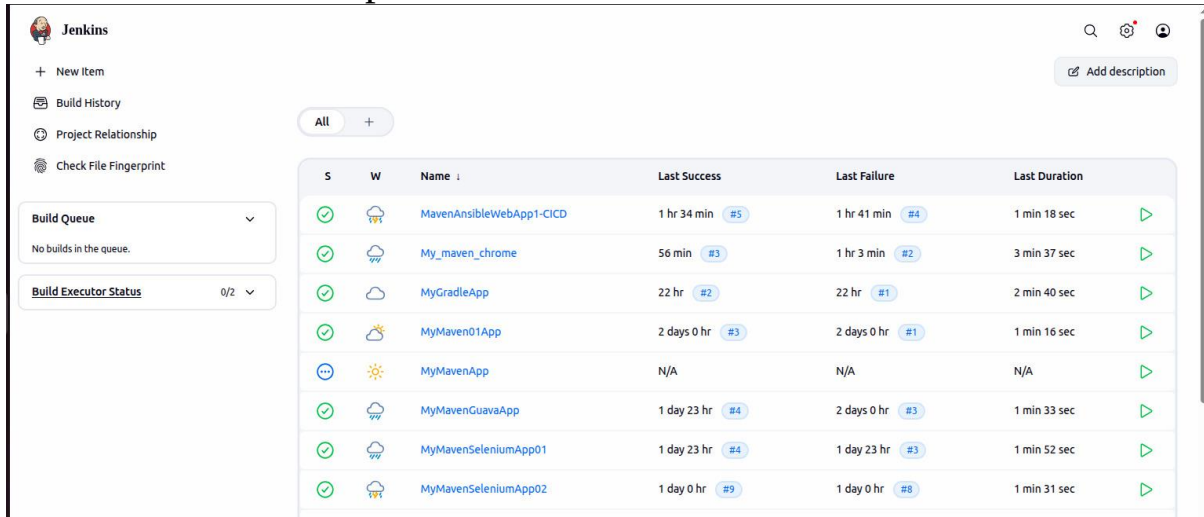
```
.
├── build.gradle
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── pom.xml
├── settings.gradle
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   └── example
│   │   │       └── App.java
│   └── test
│       ├── java
│       │   ├── com
│       │   │   └── example
│       │       └── AppTest.java
└── target
    ├── classes
    │   ├── com
    │   │   └── example
    │       └── App.class
    ├── generated-sources
    │   └── annotations
    ├── generated-test-sources
    │   └── test-annotations
    ├── maven-archiver
    │   └── pom.properties
    ├── maven-status
    │   └── maven-compiler-plugin
    │       ├── compile
    │       │   ├── default-compile
    │       │   │   ├── createdFiles.lst
    │       │   │   └── inputFiles.lst
    │       └── testCompile
    │           ├── default-testCompile
    │           │   ├── createdFiles.lst
    │           │   └── inputFiles.lst
    ├── MyMavenToGradle-1.0-SNAPSHOT.jar
    ├── surefire-reports
    │   ├── com.example.AppTest.txt
    │   └── TEST-com.example.AppTest.xml
    └── test-classes
        ├── com
        │   └── example
        │       └── AppTest.class
```


Jenkins Pipeline for MaventoGradle

Step 1: Create new Pipeline:

Go to Dashboard → + New Item.

Enter name, select Pipeline, and click OK.



The screenshot shows the Jenkins Dashboard. On the left, there's a sidebar with navigation links: '+ New Item', 'Build History', 'Project Relationship', and 'Check File Fingerprint'. Below these are two sections: 'Build Queue' (showing 'No builds in the queue.') and 'Build Executor Status' (showing '0/2'). The main area displays a table of builds with columns: 'S' (Status), 'W' (Icon), 'Name', 'Last Success', 'Last Failure', and 'Last Duration'. The table lists several builds, including 'MavenAnsibleWebApp1-CICD', 'My_maven_chrome', 'MyGradleApp', 'MyMaven01App', 'MyMavenApp', 'MyMavenGuavaApp', 'MyMavenSeleniumApp01', and 'MyMavenSeleniumApp02'. Each row shows the last successful and failed build numbers and durations.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☁	MavenAnsibleWebApp1-CICD	1 hr 34 min #5	1 hr 41 min #4	1 min 18 sec
✓	☁	My_maven_chrome	56 min #3	1 hr 3 min #2	3 min 37 sec
✓	☁	MyGradleApp	22 hr #2	22 hr #1	2 min 40 sec
✓	☁	MyMaven01App	2 days 0 hr #3	2 days 0 hr #1	1 min 16 sec
...	☀	MyMavenApp	N/A	N/A	N/A
✓	☁	MyMavenGuavaApp	1 day 23 hr #4	2 days 0 hr #3	1 min 33 sec
✓	☁	MyMavenSeleniumApp01	1 day 23 hr #4	1 day 23 hr #3	1 min 52 sec
✓	☁	MyMavenSeleniumApp02	1 day 0 hr #9	1 day 0 hr #8	1 min 31 sec

New Item

Enter an item name

MyMaventoGradle

Select an item type



The screenshot shows the 'New Item' dialog in Jenkins. It has a list of item types to choose from: 'Pipeline', 'Freestyle project', 'Maven project', 'Multi-configuration project', and 'Folder'. The 'Maven project' option is selected, indicated by a checkmark in a box. Below the list is a blue 'OK' button.

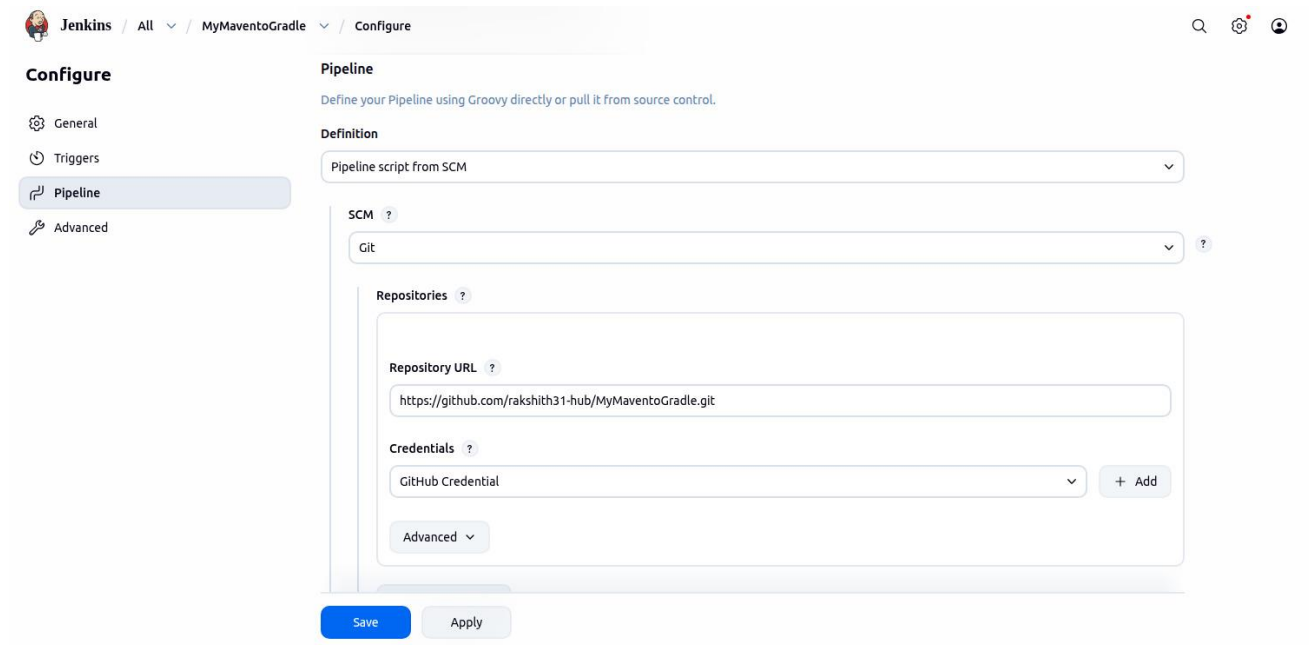
- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project** (Selected)
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**

OK

Step 2: Configure Pipeline SCM:

Scroll down and select Pipeline script from SCM.

SCM: Git Enter repository URL and select GitHub credentials.



The screenshot shows the Jenkins configuration interface for a pipeline. The breadcrumb navigation at the top indicates the path: Jenkins / All / MyMaventoGradle / Configure. On the left, the 'Configure' section is active, with sub-tabs for General, Triggers, Pipeline, and Advanced. The 'Pipeline' tab is selected. The main configuration area is titled 'Pipeline' and includes a subtitle: 'Define your Pipeline using Groovy directly or pull it from source control.' Under the 'Definition' section, 'Pipeline script from SCM' is selected. The 'SCM' section is set to 'Git'. Below this, the 'Repositories' section contains a 'Repository URL' field with the value 'https://github.com/rakshith31-hub/MyMaventoGradle.git' and a 'Credentials' dropdown menu set to 'GitHub Credential'. An '+ Add' button is next to the credentials dropdown. An 'Advanced' expandable section is also visible. At the bottom, there are 'Save' and 'Apply' buttons.

Jenkins / All / MyMaventoGradle / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

https://github.com/rakshith31-hub/MyMaventoGradle.git

Credentials

GitHub Credential + Add

Advanced

Save Apply

Step 3: Create Jenkinsfile

```
pipeline {
  agent any

  tools {
    maven 'Maven'
  }

  stages {

    stage('Checkout') {
      steps {
        git branch: 'main', url:
'https://github.com/rakshith31-hub/MyMavenToGradle.git'
      }
    }

    stage('Build') {
      steps {
        sh 'mvn clean package'
      }
    }

    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }

    stage('Check Target Folder') {
      steps {
        sh 'ls -l target'
      }
    }

    stage('Run Application') {
      steps {
        sh 'java -jar target/MyMavenToGradle-1.0-SNAPSHOT.jar'
```

```

    }
}

}

post {
    success {
        echo 'Build and execution successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
}

```

Step 4: Now Push changes and commit by pushing the necessary “Jenkinsfile” to the respective Repo

The screenshot shows the GitHub interface for a repository named 'MyMaventoGradle' by user 'rakshith31-hub'. The repository is public and has 2 branches and 0 tags. The commit history table is as follows:

Commit Message	Commit Hash	Author	Date
Update Jenkinsfile	5ef06f5	rakshith31-hub	yesterday
Update pom.xml			yesterday
gradlew.bat			3 weeks ago
gradlew			3 weeks ago
build.gradle			3 weeks ago
Jenkinsfile			yesterday
target			3 weeks ago
src			3 weeks ago
gradle/wrapper			3 weeks ago
build			3 weeks ago
.gradle			3 weeks ago

Step 5: Build a successful Pipeline

Go to Jenkins and click Build Now.

Monitor Build Stages Use Stage View and Full Stage View to monitor execution.

The screenshot shows the Jenkins web interface for a pipeline named 'MyMavenToGradle'. The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, Stages, Rename, and Pipeline Syntax. The main area displays the 'Stage View' for the selected pipeline, which is in a successful state (green checkmark). Below the stage view, there is a 'Permalinks' section with links to various build details.

Stage View

Average stage times: (full run time: ~1min 25s)

Stage	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Check Target Folder	Run Application	Declarative: Post Actions
#12 Apr 14 21:02 1 commit	3s	378ms	5s	39s	25s	798ms	1s	197ms

Permalinks

- [Last build \(#12\), 20 hr ago](#)
- [Last stable build \(#12\), 20 hr ago](#)
- [Last successful build \(#12\), 20 hr ago](#)
- [Last failed build \(#11\), 21 hr ago](#)
- [Last unsuccessful build \(#11\), 21 hr ago](#)
- [Last completed build \(#12\), 20 hr ago](#)

Builds

14 April 2026

Build	Status	Time
#12	Success	21:02
#11	Failure	20:58
#10	Failure	20:54
#9	Failure	20:51
#8	Failure	20:48
#7	Failure	20:44